Grammars

Surface syntax is important. We have discussed how the choice of lexical items (e.g., tokens or words or punctuations) makes a language more readable, and also allows us to convey intuition using some well-understood idioms and principles (such as using a + sign to denote operations that act like addition).

We have also explored how to specify the lexical items, introduced the basics of regular expressions as a language for specifying valid tokens in the language. We have also discussed using a tool such as *lex* (and variants like flex or OCaml-lex) to automatically generate a tokeniser from the lexical *specifications*. This is a robust tool, having been programmed by experts, used extremely widely for several decades, and arguably free from (all?) common bugs — certainly better than you crafting your own tokeniser. Regular expressions (whether in an abstract version or a concrete version used by tools) can be thought of a domain specific language (DSL) for front-end tokenising. Using a lex-like tool also makes it very much easier to deal with small changes in your specifications of lexemes.

However not all sequences of valid lexical tokens are well-formed phrases in a language, just as a sequence of words does not always a sentence make. For this second phase of front-end processing, we need a second abstraction, that is a specification of what sequences of tokens constitute a well-formed phrase. The mathematical abstraction used for specifying well-formed phrases is called a *Grammar*. The problem of checking whether a sequence of tokens (words, lexemes) forms a legitimate phrase is called *parsing*. Parsing is an extremely important step in linguistic analysis.

Note: In the following, we will talk about being able to *produce* a sequence of tokens using a grammar. However, in practice, we will rarely follow this top-to-down approach of producing a sequence of tokens, but rather take a *bottom-up* approach where given a sequence of tokens, we produce a justification (either explicitly as a parse tree, or implicitly as the trace of a parsing algorithm's execution) that the string can be recognised as being generated using the grammar. One typical outputs of such parsing algorithms is an abstract syntax tree (AST), which captures the essential syntactic (and therefore semantic) properties of the recognised phrase.

A Classification of Grammars

We have seen (and suffered) grammars in Natural Languages such as English, Hindi, Telugu, Sanskrit, etc. Grammars are a very useful way (notwithstanding those childhood nightmares of learning grammars, declensions etc.) for characterising sentences and other syntactic categories. For example, we can characterise simple sentences in English using the following grammar (you can make it more complicated later):

1. $S \rightarrow NP\ VP$ A sentence is a noun phrase followed by a verb phrase
2. $NP \rightarrow N$ A noun phrase can be a noun
3. $NP \rightarrow PN$ A noun phrase can be a pronoun
4. $VP \rightarrow IV$ A verb phrase can be an intransitive verb
5. $VP \rightarrow TV\ NP$ A verb phrase can be a transitive verb and then a noun phrase
6. $N \rightarrow$ Ankit A noun can be "Ankit"
7. $PN \rightarrow$ he A pronoun can be "he"

8. $IV \rightarrow$ runs An intransitive verb can be "runs"
9. $TV \rightarrow$ meets A transitive verb can be "meets"

Using these grammar rules, we can form sentences such as "he runs" or "Ankit runs or "he meets Ankit". Each of these word sequences can be show to be derivable or generated from the *Sentence* symbol *S*. A tree-shaped justification (marked with the rules used at each step) of why each of these word sequences is a sentence is called a *parse tree* (or just a parse). The grammar uses symbols such as *S*, *N*, *IV*. etc. which denote syntactic categories or phrases that are parts of a sentence, as well as lexicon items (words) such as "Ankit", "he", "runs" and "meets". We can add punctuation symbols to the lexical items.

In general, a grammar can be specified as $\mathcal{G} = \langle \mathcal{N}, \mathcal{T}, \mathcal{P}, S \rangle$, where $\mathcal{N}$ is a non-empty set of *non-terminals*, $\mathcal{T}$ is a set of lexical items called *terminals,* where $\mathcal{N} \cap \mathcal{T} = \{\}$; $S \in \mathcal{N}$ is a designated *start symbol;* finally $\mathcal{P}$ is a set of *production rules* (or grammatical rules). Each rule in $\mathcal{P}$ is of the form

$$(\mathcal{N} \cup \mathcal{T})^+ \rightarrow (\mathcal{N} \cup \mathcal{T})^*$$

that is, a non-empty sequence of symbols that are non-terminals or terminals on the left of the production arrow, and a possibly empty sequence of non-terminals and terminals on the right.

If there are no non-terminals on both sides of any production rule, the rule is a string rewriting rule; we will not discuss these any further here, other than to say that at this extreme, we have rewrite systems that are equivalent to Turing machines, and where a variety of common problems such as membership or emptiness of the language recognised become *undecidable*.

So assume that there is at least one non-terminal on the left of each production rule. A grammar is called *context-free* if the left side of each rule consists of just a single non-terminal. There may however be many productions with the same non-terminal on the left in a context-free grammar (CFG). CFGs are important for us since they constitute a relatively simple and relatively efficient model of computation (Push-down automata, which comprise a finite state control and a stack, and can be used for a class of useful computations — including producing sentences, or viewed bottom-up, recognising sentences/phrases/programs). The name "context-free" indicates the fact that we can replace a non-terminal (which appears on the left of a production rule) by its corresponding right-side of a production rule, *without* any constraints being placed by what symbols surround it.

The example grammar above is context-free. Note however that that grammar can also produce awkward word sequences such as "Ankit meets he", which is not an English sentence. The problem here is that there are other features and markers such as case, number, voice, etc. in natural languages which require some knowledge of the context. Therefore we can consider replacing left-sides of productions only when the surrounding symbols have certain formats. Such grammars, where the left side of each rule is not necessarily a single non-terminal, are called *context-sensitive*. However, while such grammars are better at filtering out the well-formed sentences from the awkward word sequences, they are more complicated to implement from a computational viewpoint (more powerful, but complexity increases; in general, some typical computation questions become harder to answer).

At the other extreme, we have a *subclass* of CFGs that are so constrained that they are no more powerful than finite state automata (FSMs/FSAs). These are the *regular grammars,* which happen to be very well-behaved in that various typical questions can be efficiently answered. We will briefly visit this class at the end of this section.

Context-Free Grammars

CFGs are a happy medium (reasonably efficient) for recognising or generating token sequences for a large class of languages, and building the corresponding abstract syntax trees. We can further filter these trees afterwards, removing those which fail some context-sensitive test, using specialised algorithmic procedures e.g., type-checking.

Let us introduce two specific examples of CFGs in the context of the programming language issues we have encountered so far.

The first is the following CFG for expressions.

1. $E \rightarrow Num$ A Numeral is an Expression
2. $E \rightarrow E{+}E$ The sum of two Expressions is an Expression
3. $E \rightarrow E{*}E$ The product of two Expressions is an Expression
4. $Num \rightarrow 0 \mid 1 \mid \ldots$ Every numeric token is a Numeral

This is a nice and simple grammar. The problem with it is that the following sequence of tokens has at least two different justifications (parse trees) for why it is an expression: 3 + 4 * 5
Such as situation is called *ambiguity*. Why is ambiguity undesirable? Because each of these different parses corresponds to different values: if parsed as (3 + 4) * 5, it evaluates to *thirty-five,* whereas if parsed as 3 + (4 * 5), it evaluates to *twenty-three*. Which parse tree is correct? Most people would think the latter meaning, because of a convention called BODMAS. But remember BODMAS is just a convention to avoid having to write too many parentheses. One could instead insist on explicitly writing parentheses

2'. $E \rightarrow (E{+}E)$ The sum of two Expressions is an Expression
3'. $E \rightarrow (E{*}E)$ The product of two Expressions is an Expression

Parentheses are useful in many situations. The make the parse explicit, and one can construct the corresponding abstract syntax tree (which has no parentheses since that information is captured by the tree structure) in a more straightforward way.

This approach of using lots of irritation silly parentheses (LISP) may solve the problem in this case, but it makes writing expressions more tedious, and can impact readability adversely.

Another pragmatic approach is to use the grammar, but externally specify that the multiplication operator binds tighter (has higher priority) than the addition operator. While this approach has its uses, it is *ad hoc*, and a lot depends on getting the priority orderings correct.

Let us try to present the same language using a more nuanced grammar, which captures in its structure the fact that multiplication binds tighter than addition:

1. $E \rightarrow T$ An expression can be a term

2. $E \rightarrow T+E$ An expression can be a term plus a expression
3. $T \rightarrow F$ A term can be a factor
4. $T \rightarrow F*T$ A term can be a factor multiplied by a term
5. $F \rightarrow Num$ A factor can be a number
6. $F \rightarrow (E)$ A factor can be an expression within parentheses
7. $Num \rightarrow 0 \mid 1 \mid \ldots$ Every numeric token is a Numeral

This is a "layered" approach that produces an unambiguous grammar, since each rule applies only in a specific case, dictated by a terminal — a numeric token, parentheses, * or +. The grammar also has the virtue of not being left-recursive (another undesirable feature); the only drawback is that it treats + and * as right associative. This is acceptable for these two operators, but may not be in general (think of subtraction and division)

Let us now turn our attention to another toy grammar, which can be modified to address a very common issue encountered in programming languages: A CFG characterising Balanced Parentheses. In the simplified form we have only the two parenthesis symbols: (and). A sequence of parenthesis is balanced if it has an equal number of left and right parentheses. The empty sequence is balanced trivially. Oh, but wait; that is not all! Every left parenthesis should precede *its* corresponding right parenthesis:)(is not balanced. But to specify which right parenthesis matches which left parenthesis requires some care. The "balanced parentheses property" is (i) that the total number of left and right parentheses in the entire sequence are equal, but also (ii) that *every* prefix of the sequence has *no more* right parentheses than left parentheses. Let's try to write such a CFG:

1. $E \rightarrow \epsilon$ The empty sequence has parentheses balanced
2. $E \rightarrow (E)$ If the inner E has parentheses balanced

But this leaves out sequences such as ()() which has balanced parentheses. So let us add the rule:

3. $E \rightarrow EE$ If the both instances of E have parentheses balanced,...

Exercise: Show by induction and case analysis on all prefixes that this grammar has the balanced parentheses property: $E \rightarrow s$ implies (i) $\#_(s = \#_) s$ and (ii) for each prefix $p \leq_{prefix} s$: $\#_(p \geq \#_) p$.

Exercise: Show by induction on the length of sequences with the balanced parentheses property, that this grammar produces all and only such sequences.

However, this grammar is horribly ambiguous: you can possibly write many different parse trees for ()(()).

Let us try to present an unambiguous CFG for balanced parentheses:

1. $E \rightarrow \epsilon$ The empty sequence has parentheses balanced
2. $E \rightarrow (E)E$ Both instances of E have parentheses balanced, and moreover...

Exercise: You may like to show that this grammar produces exactly the same strings as the previous one.

You may also like to reason why this is unambiguous. Hint: the first parenthesis, forces a use of the second rule. This grammar is also not left-recursive; it is right associative (look at the parse trees generated).

Regular Expressions as CFGs

Clearly every context-free grammar is a Context-Sensitive grammar.
Similarly every regular grammar is a CFG, where a regular grammar is characterised by requiring each production rule to be of one of the two following form:

$N \rightarrow \epsilon$ or $N \rightarrow aN'$, for some terminal a and non-terminal N'.

Clearly any grammar satisfying these more stringent conditions is trivially a CFG.

We can easily turn a (deterministic, for convenience, but w.l.o.g.) FSM into a CFG.
Let $A = \langle \Sigma, \mathcal{Q}, q, \delta, F \rangle$ be a FSM, where $q_0 \in \mathcal{Q}, \delta \subseteq \mathcal{Q} \times \Sigma \times \mathcal{Q}, F \subseteq \mathcal{Q}$.

Let us define a corresponding regular grammar $\mathcal{G} = \langle \mathcal{N}, \mathcal{T}, \mathcal{P}, S \rangle$:

- Let $\mathcal{T} \equiv \Sigma$
- For each $q_i \in \mathcal{Q}$, let there be a corresponding non-terminal $Q_i \in \mathcal{N}$
- Let start symbol $S \equiv Q_0$, the non-terminal corresponding to $q_0 \in \mathcal{Q}$
- For each $q_i \in F$, introduce a production $Q_i \rightarrow \epsilon$ into $\mathcal{P}$
- For each transition $(q_i, a, q_j) \in \delta$, put the production $Q_i \rightarrow aQ_j$ into $\mathcal{P}$.

Exercise: Show that any string $s \in \Sigma^*$ is accepted by $A = \langle \Sigma, \mathcal{Q}, q, \delta, F \rangle$ if and only if it is generated from Q_0 via the productions $\mathcal{P}$ of $\mathcal{G} = \langle \mathcal{N}, \mathcal{T}, \mathcal{P}, S \rangle$

Practical Exercise: Learn to use the tool *Yacc* or any of its variants (OCaml-yacc, Bison, .)... Here you will need to specify a grammar in a particular form (not all grammars are acceptable). The tool then *generates* a parser for you, and further, allows you to specify the corresponding semantic actions that need to be performed on *attributes* of the parse tree. At the very least, you can produce an abstract syntax tree. You can also built into the parser actions some semantic analysis — such as implementing a type-checker and further, a definitional interpreter. Thus Yacc, which stands for Yet Another Compiler Compiler, is indeed a mini-translator for a DSL that translates input grammatical specifications into parser (and beyond) code in a specified language (C in the case of the original Yacc, OCaml in the case of OCaml-yacc).

It is beyond this course to characterise these grammars. However the tool supports are some features which allow you to specify priorities of operators. Learning to use Yacc-like tools is a fundamental skill, and an important learning outcome of this course. We hope that in the future, you will rarely have to hand-craft your own parser, but will use such robust and efficient tools instead. Of course, there may be the unlikely situations where the language you wish to recognise has a peculiar grammar that is not Yacc-amenable, and you may have to hand-craft your parser. You might want to reconsider the design of that language, if that is in your control!